

Resolução de Problemas com Grafos - Caminhos Mínimos

Problema: SHPATH - The Shortest Path (SPOJ)

Grupo: Grupo C

Integrantes: Igor Praciano Thomaz, Rafael Lima Cacau e Tiago Silveira Taumaturgo

Orientador: Prof. Me Ricardo Carubbi

O Problema (SHPATH)

Contexto:

Encontrar o caminho de custo mínimo de transporte entre pares de cidades.

Desafios Principais:

- O problema lida com **strings** (nomes de cidades) em vez de números inteiros para identificar os nós.
- Cidades só podem ter no máximo 10 caracteres no nome.
- Ao todo, pode haver **múltiplos casos de teste** .

Modelagem como Grafo & Mapeamento

Vértices (V):

Cidades. Representadas internamente por **índices de `0` a `N-1`**.

Arestas Direcionadas (E):

Conexões de transporte de sentido único com **pesos não-negativos** (peso > 0, custo total acumulado 200.000).

Estruturas de Dados Utilizadas:

HashMap<String, Integer>: Mapeia o nome de cada cidade (string) para um índice inteiro único (0-based) de forma a otimizar a busca nas consultas.

EdgeWeightedDigraph (algs4): Lista de Adjacência para representar o grafo direcionado ponderado de forma eficiente.

DijkstraSP (algs4): Algoritmo de Dijkstra para caminhos mínimos a partir de uma origem única.

Exemplo de Entrada e Grafo Visualizado

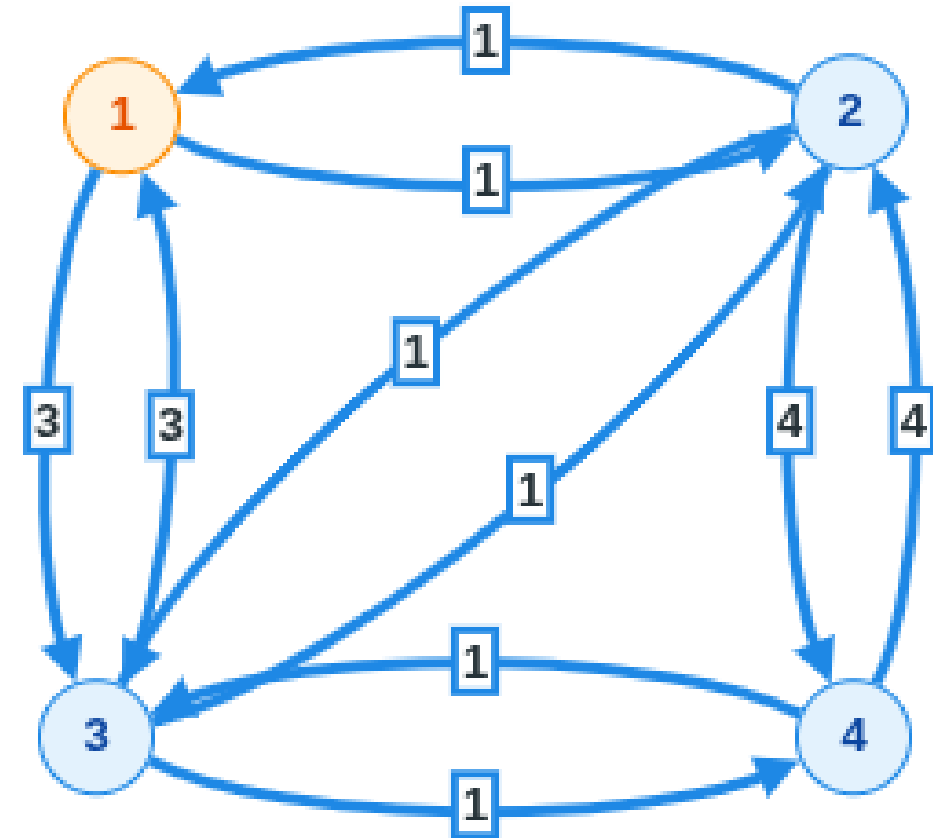
Entrada (dados02.txt)

Grafo Visualizado

Example

Input :

```
1
4
gdansk
2
2 1
3 3
bydgoszcz
3
1 1
3 1
4 4
torun
3
1 3
2 1
4 1
warszawa
2
2 4
3 1
2
gdansk warszawa
bydgoszcz warszawa
```



ESTRATÉGIA ALGORÍTMICA (O DIJKSTRA)

? Por que Dijkstra?

Os custos de transporte são estritamente maiores que zero (sem pesos negativos), o que garante a **otimalidade** e aplicabilidade do algoritmo para encontrar o caminho mais curto.

Funcionamento da Solução em Java:

01. Leitura dos vértices e mapeamento via `HashMap` para indexação rápida.
02. Adição de arestas `DirectedEdge` ajustando para indexação 0-based.
03. Para cada consulta: Instanciação de `new DijkstraSP(graph, indexOrigem)` e obtenção do custo via `.distTo(indexDestino)`.

 **Fila de Prioridades:** Utiliza-se a **IndexMinPQ** (algs4) para extrair o vértice com a menor distância provisória, garantindo a eficiência do processo de relaxamento.

COMPLEXIDADE E CASOS ESPECIAIS

Complexidade de Tempo

- Construção do Grafo: $O(V + E)$
- Dijkstra por consulta: $O((V + E) \log V)$
- Desempenho: Passa folgado nos limites do SPOJ.

Complexidade de Espaço

- Armazenamento do Grafo: $O(V + E)$
- Tabela de Hash (Cidades): $O(V)$
- Estrutura Auxiliar Dijkstra: $O(V)$

Casos Especiais Tratados

- **Destinos inalcançáveis:** Retorno de distância infinita tratado pela lógica de saída.
- **Indexação:** Conversão robusta de 1-based (entrada) para 0-based (algs4).
- **Múltiplas Consultas:** Reuso do grafo entre consultas, instanciando Dijkstra apenas para novas origens.

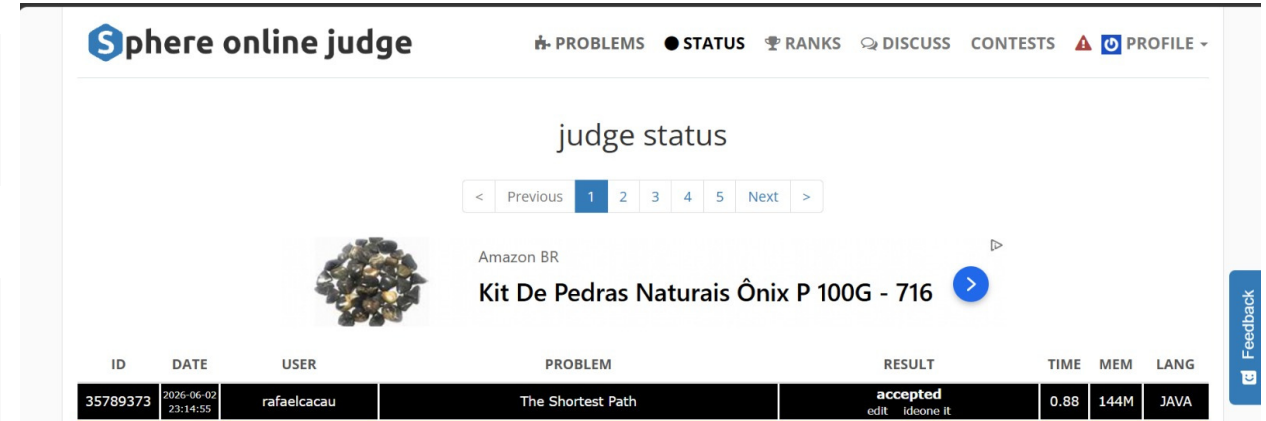
CÓDIGO JAVA E EVIDÊNCIA DE ACEITO

Mapeamento de Cidades:

```
map.put(cityName, index);  
// Otimização de busca O(1)
```

Execução do Algoritmo:

```
DijkstraSP dijkstra =  
    new DijkstraSP(graph, sourceIdx);  
double dist = dijkstra.distTo(destIdx);
```



The screenshot shows the Sphero Online Judge interface. At the top, there's a navigation bar with links: PROBLEMS, STATUS, RANKS, DISCUSS, CONTESTS, and PROFILE. The main heading is 'judge status'. Below it, there's a pagination control showing 'Previous', '1' (selected), '2', '3', '4', '5', 'Next', and '>'. A profile picture of a pile of stones is shown next to the text 'Amazon BR' and 'Kit De Pedras Naturais Ônix P 100G - 716'. Below this, there's a table with the following columns: ID, DATE, USER, PROBLEM, RESULT, TIME, MEM, and LANG. The table contains one row of data.

ID	DATE	USER	PROBLEM	RESULT	TIME	MEM	LANG
35789373	2026-06-02 23:14:55	rafaelcaca	The Shortest Path	accepted edit ideone it	0.88	144M	JAVA

Conclusão: A combinação do algoritmo de **Dijkstra** com uma fila de prioridade indexada e o mapeamento via **HashMap** permitiu resolver o problema de múltiplas rotas urbanas com eficiência máxima, respeitando todos os limites de tempo e memória da plataforma.